

Programação Orientada a Objetos



Aula 7 - Tratamento de Exceções
Prof. Guilherme Guerino



Introdução

- Erros são comuns em tempo de execução e os programas devem estar preparados para tratá-los sem abortar o programa e perder os dados dos usuários.
- Exemplos de erros:
 - Divisão por zero;
 - Valor errado de entrada (uma String no lugar de um Integer);
 - Acesso a posições inválidas em listas ou arrays.
- Tratar os erros é uma tarefa que demanda bastante codificação, tornando muitas vezes o código difícil de entender.



Introdução

- Uma exceção é um evento, normalmente associado a um erro, e que ao ser disparado interrompe o fluxo normal de execução do programa.
- Como o próprio nome indica, exceções são situações que não ocorrem com frequência.



Tratamento de Exceções

- O conceito de tratamento de exceções permite ao programador “lidar” com o problema (tratando-o) e permitindo ao programa continuar sua execução.
- O tratamento de exceções habilita os programadores a escreverem códigos robustos e tolerantes a falhas.
- Em Java é possível tratar as exceções que poderiam ocorrer para que o programa continue sua execução ou termine elegantemente.



Programação SEM Tratamento

```
package excecacao;

import java.util.Scanner;

public class Excecacao {

    public static void main(String[] args) {
        int numerador, denominador;
        Scanner leitura = new Scanner(System.in);
        System.out.println("Entre com o numerador: ");
        numerador = leitura.nextInt();
        System.out.println("Entre com o denominador: ");
        denominador = leitura.nextInt();
        System.out.println("Resultado: " + numerador/denominador);
    }
}
```



Programação SEM Tratamento

```

: Saída - Excecao (run)
run:
Entre com o numerador: 4
Entre com o denominador: 0
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at excecao.Excecao.resultado(Excecao.java:10)
    at excecao.Excecao.main(Excecao.java:20)
Java Result: 1
CONSTRUÍDO COM SUCESSO (tempo total: 4 segundos)

```



Programação SEM Tratamento

```

Saída - Excecao (run)
run:
Entre com o numerador: 4
Entre com o denominador: oi
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Scanner.java:909)
    at java.util.Scanner.next(Scanner.java:1530)
    at java.util.Scanner.nextInt(Scanner.java:2160)
    at java.util.Scanner.nextInt(Scanner.java:2119)
    at excecao.Excecao.main(Excecao.java:18)
Java Result: 1
CONSTRUÍDO COM SUCESSO (tempo total: 3 segundos)

```



Tratando as exceções

- O exemplo a seguir permite que o programa termine elegantemente se uma entrada inválida for inserida (zero para denominador ou entrada de não inteiro).



Tratando as exceções

```
package excecao;

import java.util.InputMismatchException;
import java.util.Scanner;

public class Excecao {

    public static void main(String[] args) {
        int numerador, denominador;
        Scanner leitura = new Scanner(System.in);
```



Tratando as exceções

```
try{
    System.out.println("Entre com o numerador: ");
    numerador = leitura.nextInt();
    System.out.println("Entre com o denominador: ");
    denominador = leitura.nextInt();
    System.out.println("Resultado: " + numerador/denominador);
}
catch(InputMismatchException e){
    System.err.println("Exceção: " + e);
    leitura.nextLine();
    System.out.println("Entre com valores inteiros.");
}
catch(ArithmeticException e){
    System.err.println("Exceção: " + e);
    leitura.nextLine();
    System.out.println("Entre com o denominador válido.");
}
finally{
    System.out.println("Tchau tchau.");
}
```



Modelo

```
try
{
    instruções
    instruções de aquisição de recurso
} // fim de try
catch ( UmTipoDeExceção exceção1 )
{
    instruções de tratamento de exceções
} // fim de catch
.
.
.
catch ( OutroTipoDeExceção exceção2 )
{
    instruções de tratamento de exceções
} // fim de catch
finally
{
    instruções
    instruções de liberação de recursos
} // fim de finally
```



Tratando as exceções

- Com o tratamento de exceções, o programa captura e trata (isto é, lida com) a exceção.
- A estrutura de como tratar um erro ou exceção é:
 - try - é usada para indicar um bloco de código que possa lançar (throw) uma exceção;
 - catch - serve para capturar e manipular as exceções, ou seja, tratar o erro;
 - finally - sempre será executado depois do bloco try/catch.



Atenção!

- É um erro de sintaxe colocar código entre um bloco try e seus blocos catch correspondentes.

```
try{
    System.out.println("Entre com o numerador: ");
    numerador = leitura.nextInt();
    System.out.println("Entre com o denominador: ");
    denominador = leitura.nextInt();
    System.out.println("Resultado: " + numerador/denominador);
}
int erro; Erro
catch(InputMismatchException e){
    System.err.println("Exceção: " + e);
    leitura.nextLine();
    System.out.println("Entre com valores inteiros.");
}
```



Atenção!

- Cada bloco catch pode ter apenas um único parâmetro.

```
catch(ArithmeticException e, ArithmeticException a){  
    System.err.println("Exceção: " + e);  
    leitura.nextLine();  
    System.out.println("Entre com o denominador válido.");  
}
```

Erro



Tratando as exceções e usando loop

```
package excecao;

import java.util.InputMismatchException;
import java.util.Scanner;

public class Excecao {

    public static void main(String[] args) {
        int numerador, denominador;
        Scanner leitura = new Scanner(System.in);
        boolean loop = true;
    }
}
```



Tratando as exceções e usando loop

```
do{  
    try{  
        System.out.println("Entre com o numerador: ");  
        numerador = leitura.nextInt();  
        System.out.println("Entre com o denominador: ");  
        denominador = leitura.nextInt();  
        System.out.println("Resultado: " + numerador/denominador);  
        loop = false;  
    }  
    catch(InputMismatchException e){  
        System.err.println("Exceção: " + e);  
        leitura.nextLine();  
        System.out.println("Entre com valores inteiros.");  
    }  
    catch(ArithmeticException e){  
        System.err.println("Exceção: " + e);  
        leitura.nextLine();  
        System.out.println("Entre com o denominador válido.");  
    }  
    finally{  
        System.out.println("Tchau tchau.");  
    }  
} while(loop);
```



Cláusula throws

- Passa o problema pra frente para outros o tratarem.
- Especifica as exceções que um método pode lançar.
- Aparece depois da lista de parâmetros do método e antes do corpo do método.
- Contém uma lista separada por vírgulas das exceções.
- As exceções podem ser lançadas pelas instruções no corpo do método ou pelos métodos chamados.

```
public static float resultado(int numerador, int denominador)
    throws ArithmeticException {
    return (numerador / denominador);
}
```



Cláusula throws

```
package excecao;

import java.util.InputMismatchException;
import java.util.Scanner;

public class Excecao {

    public static float resultado(int numerador, int denominador)
        throws ArithmeticException {
        return (numerador / denominador);
    }

    public static void main(String[] args) {
        int numerador, denominador;
        Scanner leitura = new Scanner(System.in);
        boolean loop = true;
    }
}
```

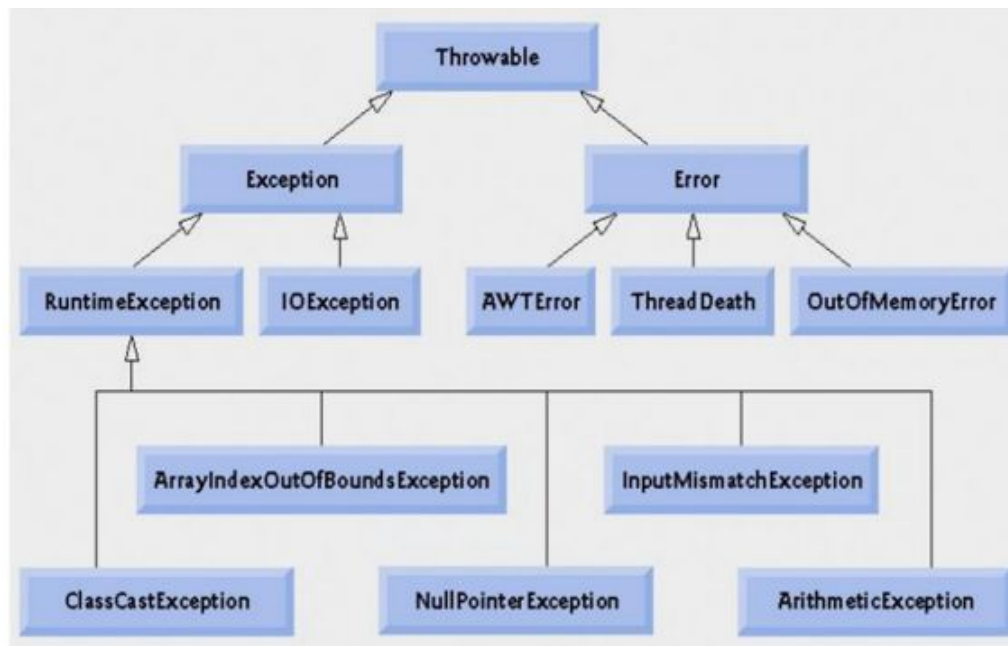


Cláusula throws

```
do {
    try {
        System.out.print("Entre com o numerador: ");
        numerador = leitura.nextInt();
        System.out.print("Entre com o denominador: ");
        denominador = leitura.nextInt();
        System.out.println("Resultado: " + resultado(numerador,
            denominador));
        loop = false;
    }
    catch (InputMismatchException e) {
        System.err.println("Exceção: " + e);
        leitura.nextLine();
        System.out.println("Entre com valores inteiros.");
    }
    catch (ArithmeticException e) {
        System.err.println("Exceção: " + e);
        leitura.nextLine();
        System.out.println("Entre com o denominador válido.");
    }
    finally {
        System.out.println("Tchau tchau");
    }
} while (loop);
```



Parte da hierarquia de exceções





Parte da hierarquia de exceções

- **Throwable** – é o pai de todas as exceções.
- **Error** – não são exceções, e sim erros que jamais poderiam ter acontecido. Exemplo: estouro da memória.
- **Exception** – as classes que derivam de Exception lançam exceções. Exemplo: tentar abrir um arquivo que não existe.
- **RuntimeException** – são exceções que indicam erros de programas (não de lógica). Esse tipo de exceção é conhecida como não verificada. Implicitamente, todas as classes em Java automaticamente já lançam uma exceção de RuntimeException.



Exceção comum

```
package excecao;

import java.util.Scanner;

public class Excecao {

    public static void main(String[] args) {
        Scanner leitura = new Scanner(System.in);
        String nome = "Marcos Domingues";
        byte posicao = 0;
        try{
            System.out.println("String: " + nome);
            System.out.println("Posição para Acessar: ");
            posicao = leitura.nextByte();
            System.out.println(nome.charAt(posicao));
        }
        catch (IndexOutOfBoundsException e){
            System.out.println("Não foi possível acessar a posição "
                               + posicao + " na String " + nome);
            System.out.println("A seguinte mensagem foi retornada: "
                               + e);
        }
    }
}
```

Dúvidas?

